

Bash Shell Scripting

Complete Open-Book Study Guide

Expanded from the 69-slide deck "**The Unix System - Bash Shell Scripting**" (CSCI 330). Every topic on every slide is covered here, with fuller explanations, traced outputs, extra examples, common mistakes, a cheat sheet and practice questions.

How to use this during the exam

- Use the **bookmarks panel** of your PDF viewer (left sidebar) to jump to any section or sub-topic.
- Use **Ctrl+F** to search any command or keyword, e.g. `trap`, `select`, `elif`, `$@`.
- Each section header lists which **original slides** it covers. The last pages hold a **cheat sheet** and a **slide index**.
- Yellow boxes show the **traced output** of an example. Red boxes warn about mistakes the slides do not mention.

Contents

1. Shell Scripts: The Basics	3
2. Variables, Arithmetic, Arrays and User Input	5
3. Special Variables and Command-Line Arguments	7
4. Decisions: if / elif / else and the test Command	9
5. Operators: Relational, Logical and File Tests	13
6. The case Statement	15
7. Repetition: while, until, for, select, break, continue	18
8. Shell Functions	24
9. Signals and Traps	27
10. Debugging Shell Programs	30
11. Extra Worked Programs for Practice	31
12. Common Mistakes and Gotchas	34
13. Quick Reference Cheat Sheet	35
14. Practice Questions with Answers	37

1. Shell Scripts: The Basics

Covers slides: 2, 3, 50, 69

A **shell script** is a plain-text file that contains shell commands. When you run it, the shell reads the file and executes the commands one after another. Like any program, a script has:

- **Data structures:** variables (plus numeric variables and arrays).
- **Control structures:** *sequence* (run top to bottom), *decision* (**if**, **case**) and *loop* (**while**, **until**, **for**, **select**).

The deck's overall outline (slide 3) is: **Input** (prompting the user, command-line arguments), **Decision** (if-then-else, case), **Repetition** (do-while / repeat-until, for, select), **Functions**, and **Traps**.

1.1 The shebang line

The first line of a script tells the operating system which interpreter should run it. It must start with **#!** (called *hash-bang* or *shebang*) followed by the path of the interpreter. A space after **#!** is allowed, so **#!/bin/bash** and **#!/bin/bash** are identical.

Shebang	Meaning
#!/bin/bash	Run with Bash (supports [[]] , read -p , arrays, select , let , local ...).
#!/bin/sh	Run with the basic POSIX shell. On Ubuntu/Debian /bin/sh is actually dash , which does NOT support read -p , [[]] , arrays or select .
#!/bin/bash -xv	Bash with debugging options turned on (see Section 11).

WATCH OUT: Several slides use **#!/bin/sh** together with **read -p** or **[[]]**. This works only where **/bin/sh** is a link to bash (e.g. older Linux, macOS). If something fails on Ubuntu, change the shebang to **#!/bin/bash**.

1.2 Making a script executable and running it

```
$ nano hello.sh          # create the file
$ chmod +x hello.sh      # make executable (slide: % chmod +x script)
$ ./hello.sh             # invoke it      (slide: % ./script)
```

The **%** in the slides is just a csh-style prompt; in bash the prompt is usually **\$**. There are other ways to run a script:

Command	What it does
./script	Runs it in a new child shell using the shebang interpreter. Needs execute permission (chmod +x).
bash script	Runs it with bash explicitly. No execute permission needed; shebang is ignored.
source script or . script	Runs it inside the current shell (variables and functions stay defined afterwards).

hello.sh

```
#!/bin/bash
# hello.sh - first script
echo "Hello, $USER"
echo "Today is $(date +%A)"
exit 0
```

Output / what happens

```
Hello, charan
Today is Thursday
```

1.3 Comments, exit status and exit

- Anything after `#` on a line is a comment (except the shebang on line 1).
- Every command returns an **exit status**: `0` = success (true), any non-zero value = failure (false). It is stored in `$?`.
- `exit n` ends the script immediately with status `n` (used in slides 20, 21 and 26 to stop on errors: `exit 1`).
- If a script has no `exit`, its status is that of the last command executed.

2. Variables, Arithmetic, Arrays and User Input

Covers slides: 4, 5, 28

2.1 Variables

- Assign with **no spaces** around `=`: `name=Ravi`. (`name = Ravi` is an error: bash thinks `name` is a command.)
- Read with `$`: `echo $name`. Use braces to separate the name from following text: `echo ${name}_file`.
- All values are strings; bash only treats them as numbers when you do arithmetic.
- Undefined variables expand to the empty string (no error by default).
- Variable names: letters, digits, underscore; cannot start with a digit; case-sensitive (`Bonus` and `bonus` differ).

Quoting	Behaviour	Example (name=Ravi)
"double"	Allows <code>\$var</code> , <code>\${cmd}</code> and backslash escapes; prevents word splitting.	<code>echo "Hi \$name" -> Hi Ravi</code>
'single'	Everything is literal.	<code>echo 'Hi \$name' -> Hi \$name</code>
\ backslash	Escapes one character.	<code>echo \\$name -> \$name</code>

Always quote variables in tests: `[ "$x" = "y" ]`. If `x` is empty and unquoted, the test becomes `[ = y ]` and fails with an error.

2.2 Command substitution

Captures a command's output into a value. The slides use backticks; the modern form `$(...)` is preferred (it nests cleanly).

```
DATE=`date +%Y%m%d`      # old style (used on slide 34)
DATE=$(date +%Y%m%d)     # modern style, same result
HOUR=$(date +%H)         # 00..23
echo "Files here: $(ls | wc -l)"
```

2.3 Numeric variables and arithmetic

The slides use `let`. Bash offers several ways to do **integer** arithmetic:

Form	Example	Notes
<code>let</code>	<code>let Net=\$Income-\$Expense / let COUNTER=\$COUNTER+1 / let i+=1</code>	No spaces around operators unless quoted.
<code>\$(())</code>	<code>Net=\$((Income - Expense))</code>	Spaces allowed; <code>\$</code> optional on variable names inside. Result can be used in <code>echo</code> .
<code>(())</code>	<code>((i++)), if ((a > b)); then</code>	Arithmetic command / test.
<code>expr</code>	<code>n=\$(expr 5 + 3)</code>	Old; <code>*</code> must be escaped as <code>*</code> .
<code>declare -i</code>	<code>declare -i n=5; n=n*2 -> 10</code>	Makes a variable integer-typed.

- Operators: `+` `-` `*` `/` `%` `**` (modulo `%`, power `**`). Division is **integer division**: `$((7/2))` is 3.
- Shorthand: `i+=1`, `i-=1`, `i++`, `i--`, `x*=2`.
- Bash has no floating point; use `bc` (`echo "7/2" | bc -l -> 3.5`) or `awk` for decimals.

2.4 Arrays (listed on slide 28)

```
fruits=(apple banana cherry)    # create
echo ${fruits[0]}               # apple      (index starts at 0)
echo ${fruits[@]}               # apple banana cherry (all elements)
echo ${#fruits[@]}              # 3          (number of elements)
fruits[3]=date                  # add/replace element
fruits+=(elderberry)            # append
for f in "${fruits[@]}"; do echo "$f"; done
```

2.5 User input with read (slides 4-5)

`read` waits for the user to type a line and stores the words in variables.

```
read varname [more vars]        # syntax, slide 4
read -p "prompt" varname [more vars] # -p shows a prompt on the same line
```

- The words typed are assigned **in order** to the variables (words are separated by spaces/tabs).
- The **last variable gets the rest of the line**. With one variable, the entire line goes into it.
- If there are more variables than words, the extra variables are empty.
- If you give no variable name, the input is stored in `$REPLY`.

Slide 5 example (works in bash)

```
#!/bin/bash
read -p "enter your name: " first last
echo "First name: $first"
echo "Last name: $last"
```

User types	first	last
Raghava Charan	Raghava	Charan
Mary Ann Smith	Mary	Ann Smith (last variable gets the rest)
Plato	Plato	(empty)

2.6 Useful read options

Option	Use
<code>-p "text"</code>	Display a prompt.
<code>-s</code>	Silent: do not echo what is typed (passwords).
<code>-n N</code>	Return after reading N characters (e.g. <code>read -n 1 key</code>).
<code>-t S</code>	Time out after S seconds.
<code>-r</code>	Raw: do not treat backslash as an escape (recommended when reading file lines).
<code>-a arr</code>	Read words into an array.

```
read -s -p "Password: " pw; echo
read -t 5 -p "Quick! Your name: " who || echo "too slow"
```

3. Special Variables and Command-Line Arguments

Covers slides: 6, 7

Arguments typed after the script name are available to the script through **positional parameters**.

Parameter	Meaning (slide 6)
<code>\$0</code>	Name of the current shell script.
<code>\$1 - \$9</code>	Positional parameters 1 through 9. Use <code>\${10}</code> for the tenth and beyond.
<code>\$#</code>	The number of positional parameters.
<code>\$*</code>	All positional parameters. "\$*" is ONE string.
<code>\$@</code>	All positional parameters. "\$@" is a set of separate strings (one per argument).
<code>\$?</code>	Return (exit) status of the most recently executed command.
<code>\$\$</code>	Process ID (PID) of the current process (the script/shell itself).

3.1 A complete example

```
#!/bin/bash
# args.sh
echo "Script name : $0"
echo "First arg   : $1"
echo "Second arg  : $2"
echo "Count      : $#"
```

```
echo "All        : $*"
echo "My PID     : $$"
```

Output / what happens

```
$ ./args.sh tim bill ann
Script name : ./args.sh
First arg   : tim
Second arg  : bill
Count      : 3
All        : tim bill ann
My PID     : 4821          (some number; differs on each run)
```

3.2 "\$*" versus "\$@" (a favourite exam question)

They differ only when quoted. Suppose the script is run as `./s.sh "a b" c` (two arguments: `a b` and `c`).

```
args() { echo "count=$#"; for a in "$@"; do echo "<$a>"; done; }
args "$*"      # passes ONE argument: "a b c"
args "$@"      # passes TWO arguments: "a b" and "c"
```

Output / what happens

```
count=1
<a b c>
count=2
<a b>
<c>
```

TIP: Use `"$@"` almost always: it preserves arguments that contain spaces. The loop `for parm` (slide 41) with no `in` part is shorthand for `for parm in "$@"`.

3.3 The set command assigns positional parameters (slide 7)

```
$ set tim bill ann fred      # $1=tim $2=bill $3=ann $4=fred
$ echo $*
tim bill ann fred
$ echo $#
4
$ echo $1
tim
$ echo $3 $4
ann fred
```

`set -- word1 word2 ...` is the safe form (the `--` stops words starting with `-` being treated as options). `set` with no arguments lists all variables.

3.4 More on arguments

- **Check the argument count first** (slide 21): `if [ $# -lt 1 ]; then echo "Usage: $0 filename"; exit 1; fi.`
- `shift` discards `$1` and moves every other argument down by one (`$2` becomes `$1`, `$#` decreases). `shift 2` shifts twice. Useful for `while [ $# -gt 0 ]` loops.
- `$_` must be read **immediately** after the command (the next command overwrites it).
- `$$` is handy for unique temporary file names: `/tmp/tempfile.$$` (slide 65).

```
ls /nonexistent 2>/dev/null
echo "status=$?"          # status=2    (non-zero = failure)
ls /etc >/dev/null
echo "status=$?"          # status=0    (success)
```

sum.sh

```
#!/bin/bash
# sum.sh - add all arguments using shift
total=0
while [ $# -gt 0 ]; do
    total=$((total + $1))
    shift
done
echo "Sum = $total"
```

Output / what happens

```
$ ./sum.sh 4 5 6
Sum = 15
```

4. Decisions: if / elif / else and the test Command

Covers slides: 8-13, 16-23

4.1 The if statement (slide 9)

```
if command
then
    statements
fi
```

The statements run only if **command succeeds, i.e. returns status 0**. Any command can be used as the condition, not only tests:

```
if grep -q "root" /etc/passwd; then
    echo "root user exists"
fi

if mkdir /tmp/mydir 2>/dev/null; then echo "created"; else echo "already exists or failed"; fi
```

4.2 The test command (slide 10)

test expression and **[expression]** are the same thing: they evaluate **expression** and return exit status 0 for true, 1 for false. **[** is really a command, so **it needs spaces**: **[-w "\$1"]**, not **[-w "\$1"]**.

Slide 10 example

```
if test -w "$1"
then
    echo "file $1 is write-able"
fi
```

Output / what happens

```
$ ./check.sh notes.txt      (notes.txt exists and is writable)
file notes.txt is write-able
$ ./check.sh /etc/passwd   (not writable by a normal user: prints nothing)
```

4.3 Simple if, if-then-else, if-elif-else (slides 11-13)

```
# Simple if
if [ condition ]; then
    statements
fi

# if-then-else
if [ condition ]; then
    statements-1          # run if condition is true
else
    statements-2          # run if condition is false
fi

# if ... elif ... else
if [ condition1 ]; then
    statements
elif [ condition2 ]; then
    statements
else
    statements
fi
```

- **;** before **then** is needed only when **then** is on the same line as **if**.
- **elif** means "else if". It is part of the **if** statement and **cannot be used by itself**. You may have many **elif** branches.

- Every **if** must be closed with **fi** ("if" backwards). Likewise **case** ends with **esac**.

4.4 Three equivalent ways to write the same test (slide 22)

```
# DOUBLE SQUARE BRACKETS
read -p "Do you want to continue?" reply
if [[ $reply = "y" ]]; then
    echo "You entered " $reply
fi

# SINGLE SQUARE BRACKETS
if [ $reply = "y" ]; then ... fi

# "TEST" COMMAND
if test $reply = "y"; then ... fi
```

Output / what happens

```
Do you want to continue?y
You entered y
```

WATCH OUT: `echo "You entered " $reply` prints two spaces because the quoted text ends with a space and `echo` adds another between arguments. With single brackets or `test`, quote the variable ("`$reply`") or an empty reply gives `[ = y ]` -> error "unary operator expected". Double brackets `[[ ]]` do not have this problem.

4.5 Worked examples from the slides

Slide 16 - using the ! (not) operator

```
#!/bin/bash
read -p "Enter years of work: " Years
if [ ! "$Years" -lt 20 ]; then
    echo "You can retire now."
else
    echo "You need 20+ years to retire"
fi
```

Output / what happens

```
Enter years of work: 25    -> You can retire now.      (25 is not less than 20)
Enter years of work: 20    -> You can retire now.      (20 is not less than 20)
Enter years of work: 12    -> You need 20+ years to retire
```

`! "$Years" -lt 20` is the same as `"$Years" -ge 20`.

Slide 17 - using && (AND)

```
#!/bin/bash
Bonus=500
read -p "Enter Status: " Status
read -p "Enter Shift: " Shift
if [[ "$Status" = "H" && "$Shift" = 3 ]]
then
    echo "shift $Shift gets \$$Bonus bonus"
else
    echo "only hourly workers in"
    echo "shift 3 get a bonus"
fi
```

Output / what happens

```

Enter Status: H
Enter Shift: 3
shift 3 gets $500 bonus          <- \$ prints a literal dollar sign, then $Bonus is replaced by 500

Enter Status: S
Enter Shift: 3
only hourly workers in
shift 3 get a bonus

```

Slide 18 - using || (OR)

```

#!/bin/bash
read -p "Enter calls handled:" CHandle
read -p "Enter calls closed: " CClose
if [[ "$CHandle" -gt 150 || "$CClose" -gt 50 ]]
then
    echo "You are entitled to a bonus"
else
    echo "You get a bonus if the calls"
    echo "handled exceeds 150 or"
    echo "calls closed exceeds 50"
fi

```

Output / what happens

```

handled=160, closed=10  -> You are entitled to a bonus    (only ONE condition needs to be true)
handled=100, closed=20  -> prints the three-line explanation (both false)

```

Slides 20 and 21 - file testing and argument checking

Slide 20

```

#!/bin/bash
echo "Enter a filename: "
read filename
if [ ! -r "$filename" ]
then
    echo "File is not read-able"
    exit 1
fi

```

If the file cannot be read, a message is printed and the script stops with status 1 (an error). Otherwise the script continues.

Slide 21

```

#!/bin/bash
if [ $# -lt 1 ]; then
    echo "Usage: filetest filename"
    exit 1
fi
if [[ ! -f "$1" || ! -r "$1" || ! -w "$1" ]]
then
    echo "File $1 is not accessible"
    exit 1
fi

```

Output / what happens

```

$ ./filetest          -> Usage: filetest filename    (no argument given; $# is 0)
$ ./filetest nosuchfile -> File nosuchfile is not accessible
$ ./filetest data.txt  -> (nothing printed; file exists, readable and writable)

```

The condition says: *not a regular file* OR *not readable* OR *not writable* -> error. Only if all three requirements hold does the script go on.

Slide 23 - if / elif / else

```
#!/bin/bash
read -p "Enter Income Amount: " Income
read -p "Enter Expenses Amount: " Expense

let Net=$Income-$Expense

if [ "$Net" -eq "0" ]; then
    echo "Income and Expenses are equal - breakeven."
elif [ "$Net" -gt "0" ]; then
    echo "Profit of: " $Net
else
    echo "Loss of: " $Net
fi
```

Output / what happens

```
Income 1000, Expenses 1000 -> Income and Expenses are equal - breakeven.
Income 1000, Expenses 400 -> Profit of: 600
Income 300, Expenses 500 -> Loss of: -200
```

5. Operators: Relational, Logical and File Tests

Covers slides: 14, 15, 19

5.1 Relational operators (slide 14)

Meaning	Numeric	String
Greater than	<code>-gt</code>	
Greater than or equal	<code>-ge</code>	
Less than	<code>-lt</code>	
Less than or equal	<code>-le</code>	
Equal	<code>-eq</code>	<code>=</code> or <code>==</code>
Not equal	<code>-ne</code>	<code>!=</code>
str1 is less than str2		<code>str1 < str2</code>
str1 is greater than str2		<code>str1 > str2</code>
String length is greater than zero		<code>-n str</code>
String length is zero		<code>-z str</code>

WATCH OUT: The slide prints the numeric "equal" operator as `-eg`; this is a typo. The correct operator is `-eq` (equal). Remember the numeric ones are letters (`-gt`, `-lt`...) and the string ones are symbols (`=`, `!=`).

- **Numbers vs strings:** `[ 10 -eq 10.0 ]` is an error (integers only), while `[ "10" = "10.0" ]` is false (different strings).
- `<` and `>` compare strings alphabetically. In single brackets they must be escaped (`[ "$a" \< "$b" ]`) or the shell treats them as redirection. In `[[ ]]` no escaping is needed.
- `[ -n "$s" ]` true if non-empty; `[ -z "$s" ]` true if empty. Always quote `$s`.
- For arithmetic comparisons you can also use `(( a > b ))`, with ordinary symbols `> >= < <= == !=`.

5.2 Compound logical expressions (slide 15)

Operator	Meaning	Notes
<code>!</code>	not	Negates the next test: <code>[! -r "\$f"]</code> .
<code>&&</code>	and	Must be inside <code>[[...]]</code> (slide 15).
<code> </code>	or	Must be inside <code>[[...]]</code> (slide 15).

```
if [[ "$age" -ge 18 && "$age" -le 60 ]]; then echo "working age"; fi      # AND
if [[ "$ans" = "y" || "$ans" = "Y" ]]; then echo "yes"; fi             # OR

# Equivalent with single brackets (two separate tests joined by the shell)
if [ "$age" -ge 18 ] && [ "$age" -le 60 ]; then echo "working age"; fi
if [ "$ans" = "y" ] || [ "$ans" = "Y" ]; then echo "yes"; fi

# Older single-bracket operators -a (and) and -o (or) work but are discouraged
if [ "$age" -ge 18 -a "$age" -le 60 ]; then echo "working age"; fi
```

5.3 [] versus [[]]

Feature	[...] / test	[[...]]
What it is	A command (works in any POSIX shell)	A bash keyword (bash/ksh/zsh only)
&& and inside	No (use -a, -o or] && [)	Yes
Unquoted empty variable	Syntax error	Safe (no word-splitting)
< > string compare	Must escape \<	Works directly
Pattern matching	No	[[\$f == *.txt]]
Regular expressions	No	[[\$s =~ ^[0-9]+\$]]

```
file="report.txt"
[[ $file == *.txt ]]    && echo "text file"      # glob match
[[ "2024" =~ ^[0-9]+$ ]] && echo "all digits"    # regex match
```

5.4 File testing (slide 19)

Test	True if...
-d file	'file' is a directory
-f file	'file' is an ordinary (regular) file
-r file	'file' is readable
-w file	'file' is writable
-x file	'file' is executable
-s file	length of 'file' is nonzero (file is not empty)
-e file	(extra) file exists, of any type
-L file	(extra) file is a symbolic link
f1 -nt f2 / f1 -ot f2	(extra) f1 is newer / older than f2

```
for item in /etc /etc/passwd /nonexistent; do
    if [ -d "$item" ]; then echo "$item is a directory"
    elif [ -f "$item" ]; then echo "$item is a regular file"
    else
        echo "$item does not exist"
    fi
done
```

Output / what happens

```
/etc is a directory
/etc/passwd is a regular file
/nonexistent does not exist
```

6. The case Statement

Covers slides: 24, 25, 26, 27

Use **case** when a decision is based on **multiple choices** of one value. It is cleaner than a long **if/elif** chain.

Syntax (slide 24)

```
case word in
  pattern1) command-list1
    ;;
  pattern2) command-list2
    ;;
  patternN) command-listN
    ;;
esac
```

6.1 Patterns (slide 25)

Each pattern is checked against **word** for a match. Patterns may contain:

Pattern element	Matches	Example
*	any string (including empty)	*.txt) any name ending .txt; *) is the default / catch-all
?	exactly one character	?) any single character
[...]	one character from a set or range	[aeiou] , [0-9] , [A-Z]
[:class:]	a character class, used inside brackets	[[:digit:]] , [[:upper:]] , [[:alpha:]]
 	separates alternative patterns	Y YES) matches Y or YES

- Each branch ends with **;;**. The statement ends with **esac** ("case" backwards).
- **The first matching pattern wins**; later patterns are not checked. So put *****) **last**.
- Matching is case-sensitive and the whole word must match (**Y** does not match **YES**).
- Unquoted **\$reply** is fine in **case** (no word-splitting problems), unlike in **[]**.

6.2 Example 1 - menu (slide 26)

```
#!/bin/bash
echo "Enter Y to see all files including hidden files"
echo "Enter N to see all non-hidden files"
echo "Enter q to quit"

read -p "Enter your choice: " reply

case $reply in
  Y|YES) echo "Displaying all (really...) files"
    ls -a ;;
  N|NO)  echo "Display all non-hidden files..."
    ls ;;
  Q)     exit 0 ;;
  *)    echo "Invalid choice!"; exit 1 ;;
esac
```

Output / what happens

```
choice Y    -> Displaying all (really...) files    + output of ls -a
choice NO   -> Display all non-hidden files...    + output of ls
choice Q    -> exits quietly with status 0
choice y    -> Invalid choice!    (lowercase does not match Y|YES)
choice q    -> Invalid choice!    (the pattern is capital Q, though the prompt says "q")
```

WATCH OUT: The slide's prompt says to enter lowercase `q`, but the pattern is `Q`). To accept both cases use bracket patterns: `[Yy][Yy][Ee][Ss]` and `[Qq]`, or convert first: `reply=$(echo "$reply" | tr '[:lower:]' '[:upper:]')`, or in bash 4+ `case ${reply^^} in`.

6.3 Example 2 - ticket rates by age (slide 27)

```
#!/bin/bash
ChildRate=3
AdultRate=10
SeniorRate=7
read -p "Enter your age: " age
case $age in
  [1-9]|[1][0-2])      # child, if age 12 and younger
    echo "your rate is" "$ChildRate.00" ;;
  # adult, if age is between 13 and 59 inclusive
  [1][3-9]|[2-5][0-9])
    echo "your rate is" "$AdultRate.00" ;;
  [6-9][0-9])          # senior, if age is 60+
    echo "your rate is" "$SeniorRate.00" ;;
esac
```

How the patterns work: patterns are matched **character by character**, so ranges of numbers are built from digit patterns.

Pattern	Matches	Ages
<code>[1-9]</code>	one digit 1-9	1 to 9
<code>[1][0-2]</code>	'1' followed by 0, 1 or 2	10, 11, 12
<code>[1][3-9]</code>	'1' followed by 3-9	13 to 19
<code>[2-5][0-9]</code>	tens digit 2-5, any units digit	20 to 59
<code>[6-9][0-9]</code>	tens digit 6-9, any units digit	60 to 99

Output / what happens

```
age 8   -> your rate is $3.00
age 12  -> your rate is $3.00
age 25  -> your rate is $10.00
age 59  -> your rate is $10.00
age 65  -> your rate is $7.00
age 0 or 100+ or letters -> no pattern matches, nothing is printed (add *) to handle errors)
```

`'$'$ChildRate.00` is just string concatenation: a literal single-quoted `$`, joined to the double-quoted value of `ChildRate` followed by `.00`.

6.4 Extra example - case with ranges of strings

```
#!/bin/bash
read -p "Enter a character: " ch
case $ch in
  [a-z])    echo "lowercase letter" ;;
  [A-Z])    echo "uppercase letter" ;;
  [0-9])    echo "digit" ;;
  ?)        echo "some other single character" ;;
  *)        echo "more than one character" ;;
esac
```

```
#!/bin/bash
# case on a file extension
for f in "$@"; do
  case "$f" in
    *.c|*.cpp) echo "$f : C/C++ source" ;;
    *.sh)      echo "$f : shell script" ;;
    *.txt)     echo "$f : text file" ;;
    *)        echo "$f : unknown type" ;;
  esac
done
```

7. Repetition: while, until, for, select, break, continue

Covers slides: 28-49, 50

Loops repeat a block of commands. The deck (slide 30) groups them into two families:

Family	Loops	How they decide to repeat
Command controlled	<code>while</code> , <code>until</code>	Run a command/test each time and look at its exit status.
List controlled	<code>for ... in</code> , <code>select</code>	Go through a list of words, one per iteration.

7.1 The while loop (slides 31-34)

Purpose: execute the commands in the command-list **as long as the expression evaluates to true**. The condition is checked **before** each pass, so the body may run zero times.

```
while [ expression ]
do
    command-list
done
```

Slide 32 - counter

```
#!/bin/bash
COUNTER=0
while [ $COUNTER -lt 10 ]
do
    echo The counter is $COUNTER
    let COUNTER=$COUNTER+1
done
```

Output / what happens

```
The counter is 0
The counter is 1
...
The counter is 9          (stops when COUNTER reaches 10)
```

Slide 33 - repeat until the user answers something other than Y

```
#!/bin/bash
Cont="Y"
while [ $Cont = "Y" ]; do
    ps -A
    read -p "want to continue? (Y/N)" reply
    Cont=`echo $reply | tr [:lower:] [:upper:]`
done
echo "done"
```

Each pass lists all processes (`ps -A`) and asks the user. `tr [:lower:] [:upper:]` converts the reply to upper case so that `y` and `Y` both mean yes. Any other answer (N, n, hello) ends the loop and prints `done`. (Safer: quote the brackets, `tr '[:lower:]' '[:upper:]'`.)

Slide 34 - webcam archiver (nested while loops)

```
#!/bin/bash
# copies files from home- into the webserver- directory
# A new directory is created every hour

PICSDIR=/home/carol/pics
WEBDIR=/var/www/carol/webcam
while true; do
    DATE=`date +%Y%m%d`
    HOUR=`date +%H`
    mkdir $WEBDIR/"$DATE"
    while [ $HOUR -ne "00" ]; do
        DESTDIR=$WEBDIR/"$DATE"/"$HOUR"
        mkdir "$DESTDIR"
        mv $PICSDIR/*.jpg "$DESTDIR"/
        sleep 3600
        HOUR=`date +%H`
    done
done
```

How it works: the outer `while true` loops forever (`true` is a command that always succeeds). Each time round it creates a directory for today's date. The inner loop runs while the hour is not `00` (midnight): it makes an hour sub-directory, moves the new `.jpg` pictures into it, sleeps one hour (3600 s) and re-reads the hour. At midnight the hour becomes `00`, the inner loop ends, and the outer loop starts again with a new date directory.

TIP: `while true; do ... done` is the standard infinite loop. Leave it with `break`, `exit` or Ctrl-C. `while :; do` also works (`:` is a no-op that succeeds).

7.2 The until loop (slides 35-37)

Purpose: execute the commands **as long as the expression evaluates to false** (i.e. repeat until it becomes true). It is the opposite of `while`.

```
until [ expression ]
do
    command-list
done
```

Slide 36 - countdown

```
#!/bin/bash
COUNTER=20
until [ $COUNTER -lt 10 ]
do
    echo $COUNTER
    let COUNTER-=1
done
```

Output / what happens

```
20
19
18
...
10
```

(when COUNTER becomes 9, the test "`9 -lt 10`" is true, so the loop stops)

Slide 37 - repeat until the user says Y

```
#!/bin/bash
Stop="N"
until [ $Stop = "Y" ]; do
    ps -A
    read -p "want to stop? (Y/N)" reply
    Stop=`echo $reply | tr [:lower:] [:upper:]`
done
echo "done"
```

	while	until
Runs the body while the test is	true (status 0)	false (non-zero)
Stops when the test becomes	false	true
<code>while [\$i -lt 10] equals</code>	<code>until [\$i -ge 10]</code>	

7.3 The for loop (slides 38-41)

Purpose: execute the commands **as many times as there are words in the argument-list**. On each pass, **variable** holds the next word.

```
for variable in argument-list
do
    commands
done
```

Slide 39 - Example 1

```
#!/bin/bash
for i in 7 9 2 3 4 5
do
    echo $i
done
```

Output / what happens

```
7
9
2
3
4
5
```

Slide 40 - Example 2

```
#!/bin/bash
# compute the average weekly temperature

for num in 1 2 3 4 5 6 7
do
    read -p "Enter temp for day $num: " Temp
    let TempTotal=TempTotal+Temp
done

let AvgTemp=TempTotal/7
echo "Average temperature: " $AvgTemp
```

The loop runs 7 times (one per day). **TempTotal** is never initialised: an unset variable counts as 0 in arithmetic, so it works, but it is better practice to write **TempTotal=0** first. **let AvgTemp=TempTotal/7** is **integer division** (total 150 -> 21, not 21.43).

Output / what happens

```
Temps 20 22 25 19 21 23 20 (total 150) -> Average temperature: 21
```

Looping over command-line arguments (slide 41)

```
#!/bin/bash
for parm
do
    echo $parm
done
```

Output / what happens

```
$ ./show.sh red green blue
red
green
blue
```

When the **in list** part is omitted, **for** iterates over the positional parameters ("\$_").

More for-loop forms

```
for i in {1..5}; do echo $i; done          # brace range: 1 2 3 4 5
for i in {0..20..5}; do echo $i; done      # with step: 0 5 10 15 20
for i in $(seq 1 2 9); do echo $i; done    # seq first step last: 1 3 5 7 9
for f in *.txt; do echo "File: $f"; done   # every .txt file in this directory
for name in "$@"; do echo "Hi $name"; done # arguments (same as plain: for name)
for u in $(cut -d: -f1 /etc/passwd | head -3); do echo $u; done # output of a command
```

```
for ((i = 0; i < 5; i++)); do echo $i; done # C-style (bash only) - familiar from C/C++
```

```
#!/bin/bash
# table of squares using the C-style loop
for ((n = 1; n <= 5; n++)); do
    echo "$n squared is $((n * n))"
done
```

Output / what happens

```
1 squared is 1
2 squared is 4
3 squared is 9
4 squared is 16
5 squared is 25
```

Reading a file line by line

```
while read -r line; do
    echo "Line: $line"
done < names.txt
```

7.4 The select command (slides 42-45)

select builds a **simple numbered menu** from a word list. The user types the **number** instead of the word. It repeats until end-of-input (**Ctrl-D**) or Ctrl-C, or until a **break/exit** in the body.

```
select WORD in LIST
do
    RESPECTIVE-COMMANDS
done
```

Slide 43

```
#!/bin/bash
select var in alpha beta gamma
do
    echo $var
done
```

Output / what happens

```
1) alpha
2) beta
3) gamma
#? 2
beta                <- typed 2, so $var = beta
#? 4
                    <- 4 is not on the menu: $var is empty, echo prints a blank line
#? 1
alpha
```

- **PS3** is the **select sub-prompt** (default `#?`). Set it to customise: `PS3="select entry or ^D: "`.
- **\$REPLY** holds the **number the user typed** (the raw input); the chosen word is in the loop variable.
- An invalid number leaves the variable empty but the loop carries on.

Slide 44

```
#!/bin/bash
PS3="select entry or ^D: "
select var in alpha beta
do
    echo "$REPLY = $var"
done
```

Output / what happens

```
1) alpha
2) beta
select entry or ^D: 2
2 = beta
select entry or ^D: 1
1 = alpha
```

Slide 45 - menu of files in the current directory

```
#!/bin/bash
echo "script to make files private"
echo "Select file to protect:"

select FILENAME in *
do
    echo "You picked $FILENAME ($REPLY)"
    chmod go-rwx "$FILENAME"
    echo "it is now private"
done
```

* expands to every file name in the current directory, so each file becomes a menu entry. After the user picks one, `chmod go-rwx` **removes read, write and execute permission from the group and others**, leaving only the owner with access.

```
#!/bin/bash
# select + case + break : a typical combination
PS3="Choose an option: "
select choice in "Show date" "Show users" "List files" "Quit"
do
    case $choice in
        "Show date") date ;;
        "Show users") who ;;
        "List files") ls ;;
        "Quit")      echo "Bye"; break ;;
        *)           echo "Invalid option $REPLY" ;;
    esac
done
```

7.5 break and continue (slides 46-49)

Both interrupt a `for`, `while` or `until` loop.

Statement	Effect
<code>break</code>	Transfers control to the statement AFTER done ; terminates the loop completely.
<code>continue</code>	Transfers control TO done / back to the loop test; skips the rest of the current iteration and continues with the next one.

Slide 47 - break

```
while [ condition ]
do
    cmd-1
    break          # this iteration is over and there are no more iterations
    cmd-n          # never reached
done
echo "done"       # execution resumes here
```

Slide 48 - continue

```
while [ condition ]
do
    cmd-1
    continue       # this iteration is over; do the next iteration
    cmd-n          # skipped
done
echo "done"
```

Slide 49

```
for index in 1 2 3 4 5 6 7 8 9 10
do
    if [ $index -le 3 ]; then
        echo "continue"
        continue
    fi
    echo $index
    if [ $index -ge 8 ]; then
        echo "break"
        break
    fi
done
```

Output / what happens

```
continue    <- index 1: <= 3, print "continue", skip the rest
continue    <- index 2
continue    <- index 3
4           <- index 4..7: pass both tests, only echo $index runs
5
6
7
8           <- index 8 is printed first ...
break       <- ... then "break" is printed and the loop ends (9 and 10 are never reached)
```

TIP: `break 2` leaves two nested loops at once; `continue 2` continues the outer loop. In a `while read` or `select` loop, `break` is the usual way to exit.

```
# stop searching as soon as the first match is found
for f in *.txt; do
    if grep -q "error" "$f"; then
        echo "First file with an error: $f"
        break
    fi
done
```

8. Shell Functions

Covers slides: 51-59

8.1 What is a function? (slides 51, 52)

- A shell function is similar to a shell script: it **stores a series of commands for execution later**.
- The shell **stores functions in memory** and runs them **in the same shell that called them** (no new process, so they can read and change your variables).
- **Where to define:** in **.profile** (always available at login), in your script, or on the command line. (**.bashrc** is also common.)
- **Remove** a function with the **unset** built-in: **unset -f funky**.
- A function **must be defined before it can be referenced**, so definitions are usually placed at the **beginning** of the script.

Syntax

```
function-name () {  
    statements  
}  
# bash also accepts: function function-name { statements; }
```

8.2 Examples (slides 53, 54)

Slide 53

```
#!/bin/bash  
  
funky () {  
    # This is a simple function  
    echo "This is a funky function."  
    echo "Now exiting funky function."  
}  
  
# declaration must precede call:  
  
funky
```

Output / what happens

```
This is a funky function.  
Now exiting funky function.
```

Slide 54

```
#!/bin/bash  
fun () { # A somewhat more complex function.  
    JUST_A_SECOND=1  
    let i=0  
    REPEATS=30  
    echo "And now the fun really begins."  
    while [ $i -lt $REPEATS ]  
    do  
        echo "-----FUNCTIONS are fun----->"  
        sleep $JUST_A_SECOND  
        let i+=1  
    done  
}  
fun
```

Prints the first message, then the line **-----FUNCTIONS are fun----->** once per second, 30 times (about 30 seconds in total). Note: calling **fun** before the definition would give "command not found".

8.3 Function parameters (slides 55-57)

- Parameters **need not be declared**.
- Arguments provided in the function call are accessible inside the function as `$1`, `$2`, `$3`, ...
- `$#` reflects the **number of parameters** (of the function call).
- `$0` **still contains the name of the script** (not the name of the function).
- `"$@"` / `$*` inside a function refer to the function's own arguments.

Slide 56

```
#!/bin/sh
testfile() {
    if [ $# -gt 0 ]; then
        if [[ -f $1 && -r $1 ]]; then
            echo $1 is a readable file
        else
            echo $1 is not a readable file
        fi
    fi
}

testfile .
testfile funtest
```

Output / what happens

<code>. is not a readable file</code>	(<code>.</code> is a directory, so <code>-f</code> fails)
<code>funtest is a readable file</code>	(if a regular readable file called <code>funtest</code> exists)
<code>funtest is not a readable file</code>	(if it does not exist)

Slide 57

```
#!/bin/bash
checkfile() {
    for file
    do
        if [ -f "$file" ]; then
            echo "$file is a file"
        else
            if [ -d "$file" ]; then
                echo "$file is a directory"
            fi
        fi
    done
}
checkfile . funtest
```

Output / what happens

<code>. is a directory</code>	(only if <code>funtest</code> exists as a regular file; a missing name prints nothing)
<code>funtest is a file</code>	

Inside the function, `for file` (no `in`) loops over **the function's arguments** `.` and `funtest`.

8.4 Local variables (slides 58, 59)

- Variables defined inside a function are **global**: their values are known throughout the entire shell program (even after the function ends).
- The keyword `local` inside a function definition makes the referenced variables local to that function.

Slide 59

```
#!/bin/bash

global="pretty good variable"

foo () {
    local inside="not so good variable"
    echo $global
    echo $inside
    global="better variable"
}

echo $global
foo
echo $global
echo $inside
```

Output / what happens

```
pretty good variable      <- echo $global (main program)
pretty good variable      <- inside foo: reads the global
not so good variable      <- inside foo: the local variable
better variable           <- back in main: foo changed the global
                           <- echo $inside prints an EMPTY line: 'inside' was local to foo
```

8.5 Returning values (extra)

Functions return an **exit status** with **return n** (0-255; 0 = success), read through **\$?**. To return text or a big number, **print it with echo** and capture it with command substitution.

```
#!/bin/bash
is_even () {
    # status-style result
    if [ $(( $1 % 2 )) -eq 0 ]; then return 0; else return 1; fi
}
add () {
    # value-style result
    echo $(( $1 + $2 ))
}

if is_even 10; then echo "10 is even"; fi
sum=$(add 3 4)
echo "3 + 4 = $sum"
```

Output / what happens

```
10 is even
3 + 4 = 7
```

```
#!/bin/bash
# recursive factorial using a local variable
fact () {
    local n=$1
    if [ $n -le 1 ]; then
        echo 1
    else
        local prev=$(fact $((n - 1)))
        echo $((n * prev))
    fi
}
echo "5! = $(fact 5)"
```

Output / what happens

```
5! = 120
```

9. Signals and Traps

Covers slides: 60-66

9.1 Handling signals (slides 60, 61)

Unix allows you to **send a signal to any process**. A signal is a small asynchronous message (e.g. "please stop"). You send one with `kill`:

Signal	Command	Meaning (slide 60)
-1 = hangup	<code>kill -HUP 1234</code>	Hang up (terminal closed); programs often reload their config.
-2 = interrupt	<code>kill -2 1235</code>	Interrupt, same as pressing <code>^C</code> .
no argument	<code>kill 1235</code>	Terminate (sends SIGTERM, signal 15).
-9 = kill	<code>kill -9 1236</code>	Kill. -9 cannot be blocked (or trapped/ignored).

- List **your** processes with `ps -u userid` (e.g. `ps -u $USER`). The PID is the first number. `ps -A` lists all processes.
- `kill -l` lists all signal names/numbers. `kill -s INT 1235` is the same as `kill -2 1235`.
- Inside a script, `$$` is its own PID, which you can print to know what to `kill` from another terminal.

Signals on Linux (slide 61)

`kill -l` shows 64 signals. The ones you should know:

No.	Name	Typical meaning / how generated
1	SIGHUP	Hangup; terminal closed
2	SIGINT	Interrupt from keyboard: <code>^C</code> is 2 - SIGINT
3	SIGQUIT	Quit from keyboard (<code>^\\</code>), makes a core dump
9	SIGKILL	Kill; cannot be caught, blocked or ignored
10 / 12	SIGUSR1 / SIGUSR2	User-defined signals
11	SIGSEGV	Invalid memory access (segmentation fault)
13	SIGPIPE	Write to a pipe with no reader
14	SIGALRM	Timer (alarm) expired
15	SIGTERM	Polite termination request (default of <code>kill</code>)
17	SIGCHLD	A child process stopped or ended
18 / 19	SIGCONT / SIGSTOP	Continue a stopped process / stop it (SIGSTOP cannot be caught)
20	SIGTSTP	Stop from keyboard (<code>^Z</code>)
34-64	SIGRTMIN ... SIGRTMAX	Real-time signals (SIGRTMIN+1 ... SIGRTMAX-1)

Slide 61's full list (1-31 standard, 34-64 real-time): SIGHUP, SIGINT, SIGQUIT, SIGILL, SIGTRAP, SIGABRT, SIGBUS, SIGFPE, SIGKILL, SIGUSR1, SIGSEGV, SIGUSR2, SIGPIPE, SIGALRM, SIGTERM, SIGSTKFLT, SIGCHLD, SIGCONT, SIGSTOP, SIGTSTP, SIGTTIN, SIGTTOU, SIGURG, SIGXCPU, SIGXFSZ, SIGVTALRM, SIGPROF, SIGWINCH, SIGIO, SIGPWR, SIGSYS, then SIGRTMIN..SIGRTMAX.

9.2 The trap command (slides 62-66)

- The **default action** for most signals is to **end the process** (the default *signal handler*).
- Bash lets you **install a custom signal handler** with `trap`.

Syntax and example

```
trap 'handler commands' signals
```

```
trap 'echo do not hangup' 1 2          # slide 62 example: for signals 1 (HUP) and 2 (INT)
```

- Put the handler in **single quotes** so variables expand when the signal arrives, not when `trap` is set up.
- Signals may be given by number (`1 2`) or name (`HUP INT, SIGINT`).
- A handler can be a command list or the name of a function: `trap 'cleanup; exit' 2`.
- Extra pseudo-signal `EXIT`: `trap 'echo bye' EXIT` runs when the script ends for any reason.
- To **ignore** a signal completely: `trap '' 2` (empty handler).
- `kill -9 (SIGKILL)` and `SIGSTOP` can never be trapped.

Slide 63 - trap hangup

```
#!/bin/bash
# kill -1 won't kill this process
# kill -2 will

trap 'echo dont hang up' 1

while true
do
    echo "try to hang up"
    sleep 1
done
```

Output / what happens

```
try to hang up
try to hang up
dont hang up      <- you ran kill -1 <pid> from another terminal: handler runs, script continues
try to hang up
...               <- kill -2 <pid> (or ^C) has no trap, so the default action ends the script
```

Slide 64 - trapping multiple signals

```
#!/bin/sh
# plain kill or kill -9 will kill this
trap 'echo 1' 1
trap 'echo 2' 2

while true; do
    echo -n .
    sleep 1
done
```

Output / what happens

```
.....1.....2...    <- dots every second; kill -1 prints 1; kill -2 (or ^C) prints 2 and the loop continues
                    Only plain kill <pid> (SIGTERM) or kill -9 <pid> stops it.
```

Slide 65 - removing temporary files on ^C

```
#!/bin/bash
trap 'cleanup; exit' 2

cleanup () {
    /bin/rm -f /tmp/tempfile.$$.*
}

for i in 1 2 3 4 5 6 7 8
do
    echo "$i.iteration"
    touch /tmp/tempfile.$$.$i
    sleep 1
done
cleanup
```

Each iteration creates a temp file named with the script's PID (\$\$) and the loop counter (tempfile.1234.1, .2, ...). If the user presses ^C (signal 2) at any time, the trap runs `cleanup` (which deletes all `/tmp/tempfile.<pid>.*` files; `*` matches the single-character suffix) and then `exits`. If the loop completes normally, the final `cleanup` call removes the files. Either way no temp files are left behind. (The function is defined after the `trap` line, which is fine because it is only **called** later.)

Slide 66 - restoring default handlers

- `trap` without a command list removes a signal handler and restores the default action.
- Use this to run a signal handler **once only**.

```
#!/bin/sh
trap 'justonce' 2
justonce() {
    echo "not yet"
    trap 2          # now reset it
}

while true; do
    echo -n "."
    sleep 1
done
```

Output / what happens

```
.....not yet.....    <- first ^C: handler prints "not yet" and resets the trap to default
.....                  <- second ^C: no handler any more, the script is terminated
```

NOTE: Modern syntax for resetting is `trap - 2` (hyphen = default action); `trap 2` as on the slide also works.

```
# more trap examples
trap 'echo "Script finished"' EXIT      # always runs at exit
trap '' INT                             # ignore ^C entirely
trap 'echo "Got SIGUSR1"' USR1          # custom signal
trap -p                                 # list current traps
```

10. Debugging Shell Programs

Covers slides: 67, 68

Debugging is troubleshooting errors that may occur while a program/script executes. Two things help (slide 67):

- `echo` - use explicit output statements to **trace execution** (print variable values and "reached here" messages).
- `set` - the built-in command that turns on shell tracing options.

10.1 Debugging using set (slide 68)

`set` is a shell built-in with options that let you watch the **flow of execution**:

Option	Effect
<code>-v</code>	Verbose : prints each line as it is read (before expansion).
<code>-x</code>	Xtrace : displays each command and its arguments after expansion, prefixed with <code>+</code> .
<code>-n</code>	No-exec : checks for syntax errors only; does not run the script.

- Options can be turned **on or off**: turn on with `set -xv`; turn off with `set +xv`. (A minus *enables*, a plus *disables*: it is counter-intuitive.)
- Options can also be set via the **she-bang line**: `#!/bin/bash -xv`.
- Or from the command line without editing: `bash -x script.sh`, `bash -n script.sh`.

```
#!/bin/bash
x=5
set -x          # start tracing here
y=$((x + 1))
echo "y is $y"
set +x         # stop tracing
echo "done"
```

Output / what happens

```
+ y=6          <- trace lines (printed to stderr), values already expanded
+ echo 'y is 6'
y is 6         <- the real output
+ set +x
done
```

10.2 Other helpful settings (extra)

Setting	Effect
<code>set -e</code>	Exit immediately if any command fails (non-zero status).
<code>set -u</code>	Treat use of an unset variable as an error (catches typos like <code>\$Conter</code>).
<code>set -o pipefail</code>	A pipeline fails if any command in it fails.
<code>bash -n script</code>	Syntax check only: finds missing <code>fi</code> , <code>done</code> , <code>esac</code> .
<code>echo "DEBUG: x=\$x" >&2</code>	Print debug info to stderr so it does not mix with normal output.

TIP: Typical syntax errors: `unexpected end of file` = a missing `fi/done/esac`; `[: too many arguments` = unquoted variable with spaces; `unary operator expected` = unquoted empty variable; `command not found` = space around `=` or function not defined yet.

11. Extra Worked Programs for Practice

Complete scripts that combine the ideas above. Try tracing each one by hand before reading the output.

11.1 Even or odd (if + arithmetic)

```
#!/bin/bash
read -p "Enter a number: " n
if [ $((n % 2)) -eq 0 ]; then
    echo "$n is even"
else
    echo "$n is odd"
fi
```

11.2 Grade calculator (if / elif)

```
#!/bin/bash
read -p "Enter marks (0-100): " m
if [ $m -ge 90 ]; then echo "Grade A"
elif [ $m -ge 75 ]; then echo "Grade B"
elif [ $m -ge 50 ]; then echo "Grade C"
else
    echo "Fail"
fi
```

11.3 Simple calculator (case + arguments)

```
#!/bin/bash
# usage: ./calc.sh 8 + 3
if [ $# -ne 3 ]; then echo "Usage: $0 num op num"; exit 1; fi
case $2 in
    +) echo $(( $1 + $3 )) ;;
    -) echo $(( $1 - $3 )) ;;
    x|x|' * ') echo $(( $1 * $3 )) ;;
    /) if [ $3 -eq 0 ]; then echo "Cannot divide by zero"; exit 1; fi
        echo $(( $1 / $3 )) ;;
    *) echo "Unknown operator $2"; exit 1 ;;
esac
```

Output / what happens

```
$ ./calc.sh 8 + 3      -> 11
$ ./calc.sh 8 x 3      -> 24
$ ./calc.sh 8 / 0      -> Cannot divide by zero
```

11.4 Guessing game (while + break + read)

```
#!/bin/bash
secret=$((RANDOM % 10 + 1))      # RANDOM is a built-in random number 0-32767
tries=0
while true; do
    read -p "Guess (1-10): " g
    let tries++
    if [ "$g" -lt $secret ]; then echo "Too low"
    elif [ "$g" -gt $secret ]; then echo "Too high"
    else echo "Correct in $tries tries!"; break
    fi
done
```

11.5 Multiplication table (nested for)

```
#!/bin/bash
for i in 1 2 3; do
    for j in 1 2 3 4 5; do
        echo -n "${i} * ${j} "
    done
    echo
done
```

Output / what happens

```
1 2 3 4 5
2 4 6 8 10
3 6 9 12 15
```

11.6 Prime test (function + loop + return)

```
#!/bin/bash
is_prime () {
    local n=$1
    [ $n -lt 2 ] && return 1
    for ((i = 2; i * i <= n; i++)); do
        [ $(n % i) -eq 0 ] && return 1
    done
    return 0
}
for k in 1 2 7 10 13; do
    if is_prime $k; then echo "$k is prime"; else echo "$k is not prime"; fi
done
```

Output / what happens

```
1 is not prime
2 is prime
7 is prime
10 is not prime
13 is prime
```

11.7 File report (for over arguments + file tests + case)

```
#!/bin/bash
if [ $# -eq 0 ]; then echo "Usage: $0 file..."; exit 1; fi
for f in "$@"; do
    if [ ! -e "$f" ]; then echo "$f: does not exist"; continue; fi
    if [ -d "$f" ]; then type="directory"
    elif [ -f "$f" ]; then type="regular file"
    else
        type="other"
    fi
    perms=""
    [ -r "$f" ] && perms="${perms}r"
    [ -w "$f" ] && perms="${perms}w"
    [ -x "$f" ] && perms="${perms}x"
    echo "$f: $type, permissions for you: $perms"
done
```

11.8 Safe backup with trap cleanup

```
#!/bin/bash
tmp=tmp/backup.$$
cleanup () { rm -rf "$tmp"; echo "cleaned up"; }
trap 'cleanup; exit 1' INT TERM      # on ^C or kill
trap cleanup EXIT                  # on normal exit as well

mkdir "$tmp"
cp -r "$1" "$tmp"/ || { echo "copy failed"; exit 1; }
tar -czf "backup_$(date +%Y%m%d).tar.gz" -C "$tmp" .
echo "backup done"
```

11.9 Count lines in every text file (while read + counters)

```
#!/bin/bash
total=0
for f in *.txt; do
    [ -f "$f" ] || continue          # skip if the glob matched nothing
    n=$(wc -l < "$f")
    echo "$f: $n lines"
    total=$((total + n))
done
echo "Total: $total lines"
```

11.10 Menu-driven script (select + case + functions)

```
#!/bin/bash
show_disk () { df -h | head -5; }
show_mem  () { free -m; }
PS3="Enter choice: "
select opt in "Disk usage" "Memory" "Quit"; do
    case $REPLY in
        1) show_disk ;;
        2) show_mem ;;
        3) break ;;
        *) echo "Pick 1, 2 or 3" ;;
    esac
done
```

12. Common Mistakes and Gotchas

Mistake	What happens	Fix
<code>x = 5</code> (spaces around =)	<code>x: command not found</code>	<code>x=5</code>
<code>[\$a -eq 1]</code> with empty <code>\$a</code>	unary operator expected	<code>["\$a" -eq 1]</code> or <code>[[]]</code>
<code>[\$a -eq 1]</code> (no spaces)	<code>[5: command not found</code>	<code>["\$a" -eq 1]</code>
<code>if [\$a -eq 1] then</code> (no ;/newline)	syntax error	<code>if [...]; then</code>
<code>&&/ </code> inside <code>[]</code>	syntax error / wrong result	Use <code>[[]]</code> or <code>[] && []</code>
<code>[\$a > \$b]</code>	creates a file named <code>b!</code>	<code>[[\$a > \$b]]</code> or <code>["\$a" \> "\$b"]</code>
<code>-eg</code> instead of <code>-eq</code>	integer expression expected	Use <code>-eq</code>
Missing <code>fi</code> / <code>done</code> / <code>esac</code>	unexpected end of file	Close every <code>if</code> , loop and <code>case</code>
Forgetting <code>;;</code> in <code>case</code>	syntax error	End every branch with <code>;;</code>
Calling a function before defining it	command not found	Define functions first
Expecting function variables to be local	They are global	Use <code>local var</code>
<code>let i = i + 1</code>	error (spaces)	<code>let i=i+1</code> or <code>((i++))</code> or <code>i=\$((i+1))</code>
Using <code>/bin/sh</code> with bash features	<code>read -p, [[]]</code> fail on dash	Use <code>#!/bin/bash</code>
<code>\$@</code> unquoted	arguments with spaces get split	<code>"\$@"</code>
Reading <code>\$?</code> late	It holds the status of the last command only	Save it: <code>rc=\$?</code>
Floating point <code>\$((7/2))</code>	gives 3	Use <code>bc</code> / <code>awk</code>
Hard-coded <code>case</code> letters in one case only	<code>y</code> not matching <code>Y</code>	<code>[Yy]</code> patterns
Forgetting <code>chmod +x</code>	Permission denied	<code>chmod +x script</code> or <code>bash script</code>

13. Quick Reference Cheat Sheet

Structure at a glance

```
#!/bin/bash                                # shebang
var=value ; echo "$var"                    # variables (no spaces around =)
read -p "prompt: " a b                     # input
let n=n+1  n=$((n+1))  ((n++))             # integer arithmetic
out=$(command)                             # command substitution

if [ cond ]; then ... elif [ cond ]; then ... else ... fi
case $x in p1|p2) ... ;; [a-z]) ... ;; *) ... ;; esac
while [ cond ]; do ... done                # repeat while true
until [ cond ]; do ... done                # repeat until true
for v in list; do ... done                 # list loop (for v = over "$@")
for ((i=0;i<n;i++)); do ... done           # C-style
select v in list; do ... done              # numbered menu (PS3 prompt, $REPLY)
break continue                            # leave loop / next iteration
name () { ...; return 0; }                 # function; local var; $1 $2 $# inside
trap 'cmds' SIGNALS ; trap - SIG          # set / reset signal handler
set -x set -v set -n                      # debug (use + to turn off)
exit n                                     # end script with status n
```

Special variables

Var	Meaning	Var	Meaning
\$0	script name	\$#	number of arguments
\$1..\$9	positional params	\$?	last exit status
\$*	all args (one string if quoted)	\$\$	PID of the shell/script
\$@	all args (separate if quoted)	\$REPLY	default read / select input

Test operators

Numeric	String	File	Logic
-eq equal	= == equal	-e exists	! not
-ne not equal	!= not equal	-f regular file	&& and (in [[]])
-gt greater	< less (alphabetical)	-d directory	or (in [[]])
-ge greater/equal	> greater	-r readable	-a and (in [])
-lt less	-n s non-empty	-w writable	-o or (in [])
-le less/equal	-z s empty	-x executable; -s size>0	

Signals and kill

Command	Effect
kill pid	SIGTERM (15): polite terminate
kill -1 pid / kill -HUP pid	SIGHUP: hangup
kill -2 pid / ^C	SIGINT: interrupt
kill -9 pid	SIGKILL: cannot be blocked or trapped
ps -u userid	list your processes

Debug options

Option	Effect	Option	Effect
<code>set -v</code>	print lines as read	<code>set +v</code>	turn it off
<code>set -x</code>	print commands with args	<code>set +x</code>	turn it off
<code>set -n</code>	syntax check only	<code>#!/bin/bash -xv</code>	set from the shebang

14. Practice Questions with Answers

Try them first, then read the answers.

Part A - Short answer

- **Q1.** What does the first line `#!/bin/bash` do? **A:** It tells the kernel to run the script with the bash interpreter.
- **Q2.** Command to make `script` executable and then run it? **A:** `chmod +x script` then `./script`.
- **Q3.** What do `$#`, `$?` and `$$` hold? **A:** Number of arguments; exit status of the last command; PID of the current process.
- **Q4.** Difference between `"$"` and `"$@"`? **A:** `"$"` expands to one single string of all arguments; `"$@"` expands to a separate string per argument.
- **Q5.** What does `if command; then ...` consider "true"? **A:** An exit status of 0.
- **Q6.** Where must `&&` and `||` be placed when combining conditions on slide 15? **A:** Inside double square brackets `[[ ... ]]`.
- **Q7.** Difference between `while` and `until`? **A:** `while` repeats while the expression is true; `until` repeats while it is false.
- **Q8.** Which two loops are *command controlled* and which two are *list controlled*? **A:** Command: `while`, `until`. List: `for ... in`, `select`.
- **Q9.** What is `PS3` and what is `$REPLY` in `select`? **A:** `PS3` is the select prompt; `$REPLY` is the number the user typed.
- **Q10.** What is the difference between `break` and `continue`? **A:** `break` leaves the loop; `continue` skips to the next iteration.
- **Q11.** Are variables set inside a function local? **A:** No, they are global unless declared with `local`.
- **Q12.** Why define functions at the top of a script? **A:** A function must be defined before it is called.
- **Q13.** Which signals can never be trapped? **A:** `SIGKILL` (9) and `SIGSTOP`.
- **Q14.** How do you remove a trap so that the default action returns? **A:** `trap signal` (or `trap - signal`) with no command list.
- **Q15.** What do `set -v`, `set -x`, `set -n` do? **A:** Print each line as read; print each command with its arguments; check syntax without executing.
- **Q16.** How to turn off debug options turned on with `set -xv`? **A:** `set +xv`.

Part B - What is the output?

```
# B1
for i in 1 2 3; do
    if [ $i -eq 2 ]; then continue; fi
    echo $i
done
```

Answer: 1 and 3 (the number 2 is skipped).

```
# B2
COUNTER=3
until [ $COUNTER -lt 1 ]; do
    echo $COUNTER
    let COUNTER-=1
done
```

Answer: 3, 2, 1 on separate lines.

```
# B3
set a b c d
echo $# $2 $*
shift
echo $# $1
```

Answer: 4 b a b c d then 3 b.

```
# B4
x=10
f () { local x=5; echo $x; }
f
echo $x
```

Answer: 5 then 10 (the local x does not touch the global one).

```
# B5
reply=Y
case $reply in
  y|yes) echo one ;;
  Y|YES) echo two ;;
  *)     echo three ;;
esac
```

Answer: two.

```
# B6
for index in 1 2 3 4 5
do
    if [ $index -ge 4 ]; then break; fi
    echo $index
done
echo "done"
```

Answer: 1, 2, 3, done.

Part C - Write the script

- **C1.** Read a number from the user and print whether it is positive, negative or zero. *Hint:* if / elif / else with -gt, -lt.
- **C2.** Take a filename as argument; print an error and exit with 1 if no argument is given, or if the file is not readable; otherwise print its number of lines. *Hint:* \$#, -r, wc -l.
- **C3.** Print the sum of numbers 1 to N where N is read from the user (while loop).
- **C4.** A menu with select offering Date, Users, Quit; Quit must stop the loop.
- **C5.** A script that loops forever printing the time every second, but on ^C prints "Bye" and exits (use trap).

```
# C1 answer
read -p "Number: " n
if [ $n -gt 0 ]; then echo "positive"
elif [ $n -lt 0 ]; then echo "negative"
else echo "zero"
fi

# C3 answer
read -p "N: " N; i=1; sum=0
while [ $i -le $N ]; do sum=$((sum + i)); let i++; done
echo "Sum = $sum"

# C5 answer
trap 'echo "Bye"; exit 0' INT
while true; do date +%T; sleep 1; done
```

15. Slide Index (where each slide is covered)

Slides	Topic	Section
1	Title: The Unix System - Bash Shell Scripting	-
2	Basic shell programming: script, shebang, chmod +x, ./script	1
3	Bash outline: input, decision, repetition, functions, traps	1
4, 5	User input: read, read -p; example (first/last)	2.5
6	Special shell variables (\$0, \$1-\$9, \$#, \$*, \$@, \$?, \$\$)	3
7	Command-line argument examples with set	3.3
8	Control structures: if-then-else, case, loops	4, 6, 7
9-13	if, test command, simple if, if-then-else, if...elif	4.1 - 4.3
14	Relational operators (numeric / string)	5.1
15	Compound logical expressions (!, &&,)	5.2
16-18	Examples using !, &&,	4.5
19-21	File testing operators and examples	5.4, 4.5
22	Single [], double [[]] brackets and test	4.4
23	if..elif example (profit/loss)	4.5
24-27	case statement, patterns, two examples	6
28, 29	Summary: so far / still to come	1
30	Repetition constructs (command / list controlled)	7
31-34	while loop and examples	7.1
35-37	until loop and examples	7.2
38-41	for loop, examples, looping over arguments	7.3
42-45	select command, PS3, \$REPLY, examples	7.4
46-49	break and continue	7.5
50	Progress summary (functions, traps still to come)	1
51-59	Shell functions, parameters, local variables	8
60-61	Handling signals, signals on Linux	9.1
62-66	trap: syntax, multiple signals, temp files, restoring defaults	9.2
67-68	Debugging with echo and set (-v, -x, -n)	10
69	Summary: everything DONE	-